Reviewer Pack — Minimal Rank of Noncommutative Crossed Products Bounds XOR-A...

Entry 5760cad45dbf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 03:48:00 UTC

Minimal Rank of Noncommutative Crossed Products Bounds XOR-AND Tree Width

Entry ID: `5760cad45dbf` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 03:48:00 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Algebra (Crossed Products) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

> {'sentence_1': 'The minimal rank of the noncommutative crossed product algebra associated with a given XOR-AND tree T is upper-bounded by a function f(n) = O(log n log log n).', 'sentence_2': 'For all instances of XOR-AND trees, if the minimal rank is less than or equal to f(n), then the width of the tree is at most 2^n.', 'sentence_3': 'Conversely, there exists an XOR-AND tree with a minimal rank greater than f(n) and a width greater than 2^n.'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Noncommutative crossed products provide a natural algebraic framework to study the complexity of combinatorial objects like XOR-AND trees.', 'sentence_2': 'The minimal rank of a noncommutative crossed product captures information about the structure of the algebra, which may be related to the width of the tree.', 'sentence_3': 'By linking these two fields, we may uncover new structural insights into the complexity of XOR-AND trees and potentially find tighter bounds.'}

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f6c2125cce68f0fd`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> If the minimal rank of noncommutative crossed product algebras for all XOR-AND trees up to n=40 is less than or equal to f(n) = O(log n log log n), and at least 90% of seeds have a mean metric value ≤ 2^n.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - 'noncommutative crossed product algebra' AND 'XOR-AND tree width' - 'minimal rank' AND 'crossed product algebra' AND 'tree width complexity' - 'XOR-AND tree width' AND 'upper bound minimal rank noncommutative algebra'

Top relevant hits considered: - [http://arxiv.org/abs/0804.4584v1] Feature Unification in TAG Derivation Trees - [http://arxiv.org/abs/cs/0406024v1] Layout of Graphs with Bounded Tree-Width - [http://arxiv.org/abs/1412.6201v1] An Upper Bound on the Size of Obstructions for Bounded Linear Rank-Width

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def generate_tree(n):
    if n == 1:
        return [0, 1]
    left = generate_tree(n-1)
    right = generate_tree(n-1)
    return [[left[i] ^ right[i] for i in range(2**(n-1))], [left[i] & right[i] for i in range(2**(
```

```python
def compute_noncommutative_crossed_product(tree):
    n = len(tree[0])
    if n == 1:
        return [[tree[0][0]], [tree[1][0]]]

    left = compute_noncommutative_crossed_product(tree[0])
    right = compute_noncommutative_crossed_product(tree[1])

    new_left = []
    new_right = []
    for i in range(2**(n-1)):
        for j in range(2**(n-1)):
            new_left.append(left[i][j] ^ right[j][i])
            new_right.append(left[i][j] & right[j][i])
```

```
    return [new_left, new_right]

def compute_minimal_rank(algebra):
    n = len(algebra[0])
    rank = 0
    for i in range(n):
        if algebra[0][i] != 0 or algebra[1][i] != 0:
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        tree = generate_tree(n)
        algebra = compute_noncommutative_crossed_product(tree)
        minimal_rank = compute_minimal_rank(algebra)

        f_n = math.log(n) * math.log(math.log(n))
        if minimal_rank <= f_n:
            width = 2**n
            if width > 2**n:
                return {
                    "metric_name": "width",
                    "metric_value": width,
                    "instances_tested": n,
                    "conjecture_holds": False,
                    "counterexample": "Tree with minimal rank <= f(n) but width > 2^n"
                }
        else:
            if minimal_rank > f_n and width > 2**n:
                return {
                    "metric_name": "width",
                    "metric_value": width,
                    "instances_tested": n,
                    "conjecture_holds": False,
                    "counterexample": "Tree with minimal rank > f(n) but width > 2^n"
                }

    return {
        "metric_name": "width",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {result}")
        results.append(result["metric_value"])

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean)**2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r <= 2**n) / len(results)

    if all(r <= 2**n for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(r > 2**n for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if result > 2**n)
        print(f"RESULT: FALSIFIED counterexample=\"Tree with minimal rank <= f(n) but width > 2^n\
    else:
        print("RESULT: INCONCLUSIVE reason=metric_saturation")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3012c98e.py", line 94, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3012c98e.py", line 55, in run_trial
    tree = generate_tree(n)
           ^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3012c98e.py", line 21, in generate_tree
    left = generate_tree(n-1)
           ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3012c98e.py", line 21, in generate_tree
    left = generate_tree(n-1)
           ^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3012c98e.py", line 23, in generate_tree
    return [[left[i] ^ right[i] for i in range(2**(n-1))], [left[i] & right[i] for i in range(2**(n-
1))]]
            ~~~~~~~~^~~~~~~~~
TypeError: unsupported operand type(s) for ^: 'list' and 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that it was unable to complete the required computations to verify the conjecture. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17416
2	preregistration	ollama_remote	glm4:latest	0	0	5615
3	novelty	ollama_remote	glm4:latest	0	0	4795
4	novelty	ollama_remote	glm4:latest	0	0	5983
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19427
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11612
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11814
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10707
9	judge	ollama_remote	glm4:latest	0	0	25638

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 113008 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/5760cad45dbf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5760cad45dbf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5760cad45dbf.tar.gz` (if generated)